Assignment 1

```
Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Information Security
```

Part A : Password Strength and Cracking

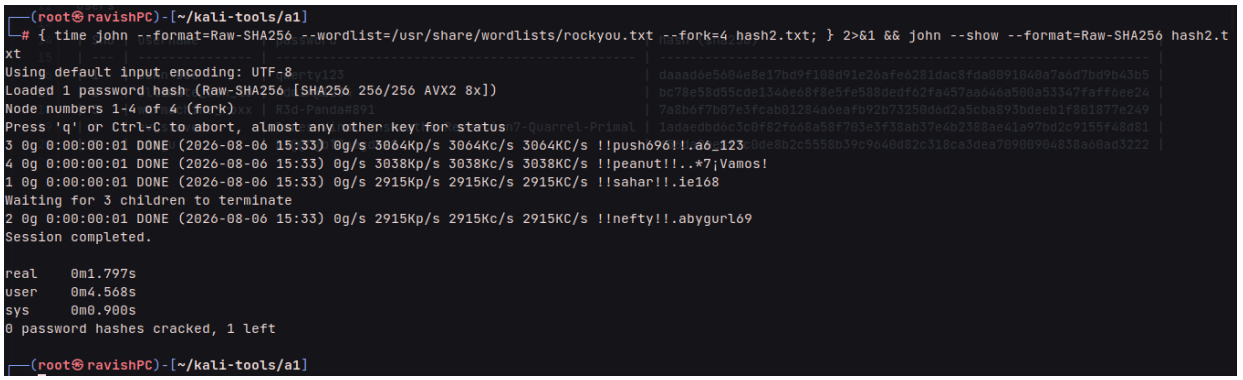
Users

username	password	hash (sha256)
John Doe	qwerty123	daaad6e5604e8e17bd9f108d91e26afe6281dac8fda0091040a7a6d7bd9b43b5
ultimate_ben_10	Admin@1234	bc78e58d55cde1346e68f8e5fe588dedf62fa457aa646a500a53347faff6ee24
warmachine_roxx	R3d-Panda#891	7a8b6f7b07e3fcab01284a6eafb92b73250d6d2a5cba893bdeeb1f801877e249
ev_stevee	Dares-Viral-Unsheathe-Reversion7-Quarrel-Primal	1adaedbd6c3c0f82f668a58f703e3f38ab37e4b2388ae41a97bd2c9155f48d81
pingu	h1@@C@bTJamgdec2	456da2ee8f7c0de8b2c5558b39c9640d82c318ca3dea70900904838a60ad3222

1.
   - With wordlist



2.
   - With wordlist



- With incremental approach

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# { time john --format=Raw-SHA256 --incremental --fork=4 hash2.txt; } 2>&1 && john --show --format=Raw-SHA256 hash2.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
2 0g 0:00:02:11  0g/s 5220Kp/s 5220Kc/s 5220KC/s 21264pis..21267tix
3 0g 0:00:02:11  0g/s 5255Kp/s 5255Kc/s 5255KC/s jhylb0w..jhysp97
1 0g 0:00:02:11  0g/s 5240Kp/s 5240Kc/s 5240KC/s rh017442..rh017as3
4 0g 0:00:02:10  0g/s 5250Kp/s 5250Kc/s 5250KC/s 8loqsd..8le8lk
2 0g 0:00:04:06  0g/s 5239Kp/s 5239Kc/s 5239KC/s klmairmy..klmags37
3 0g 0:00:04:06  0g/s 5250Kp/s 5250Kc/s 5250KC/s jciizón..jcii122
1 0g 0:00:04:06  0g/s 5154Kp/s 5154Kc/s 5154KC/s wkndi9$..wkndsai
4 0g 0:00:04:05  0g/s 5206Kp/s 5206Kc/s 5206KC/s ivm12/65..ivm12.HE
1 0g 0:00:10:03  0g/s 5099Kp/s 5099Kc/s 5099KC/s 152b3cse..152b3317
Waiting for 3 children to terminate
4 0g 0:00:10:03  0g/s 5134Kp/s 5134Kc/s 5134KC/s dshb05jg..dshb09ak
3 0g 0:00:10:03  0g/s 5099Kp/s 5099Kc/s 5099KC/s wmjnlnm..wmjnu1@
2 0g 0:00:10:03  0g/s 5089Kp/s 5089Kc/s 5089KC/s ciegolvha1..ciegodavy2
Session aborted

real    10m3.908s
user    37m3.439s
sys     0m2.083s

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# john --show --format=Raw-SHA256 hash2.txt
0 password hashes cracked, 1 left

┌──(root㉿ravishPC)-[~/kali-tools/a1]
```

3. - With wordlist

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# { time john --format=Raw-SHA256 --wordlist=/usr/share/wordlists/rockyou.txt --fork=4 hash3.txt; } 2>&1 && john --show --format=Raw-SHA256 hash3.t
xt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
4 0g 0:00:00:01 DONE (2026-08-06 15:35) 0g/s 3415Kp/s 3415Kc/s 3415KC/s !!peanut!!..*7¡Vamos!
1 0g 0:00:00:01 DONE (2026-08-06 15:35) 0g/s 3289Kp/s 3289Kc/s 3289KC/s !!sahar!!.ie168
Waiting for 3 children to terminate
2 0g 0:00:00:01 DONE (2026-08-06 15:35) 0g/s 3289Kp/s 3289Kc/s 3289KC/s !!nefty!!.abygurl69
3 0g 0:00:00:01 DONE (2026-08-06 15:35) 0g/s 3230Kp/s 3230Kc/s 3230KC/s !!push696!!.a6_123
Session completed.

real    0m1.675s
user    0m4.570s
sys     0m0.926s
0 password hashes cracked, 1 left

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─#
```

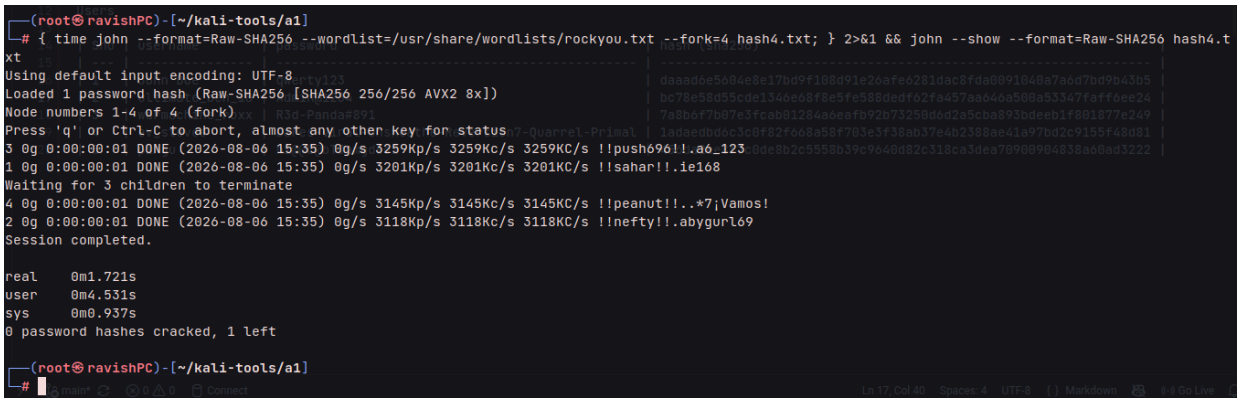
- With incremental approach

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# { time john --format=Raw-SHA256 --incremental --fork=4 hash3.txt; } 2>&1 && john --show --format=Raw-SHA256 hash3.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
1 0g 0:00:02:18  0g/s 4398Kp/s 4398Kc/s 4398KC/s peedylino..peedycole
2 0g 0:00:02:18  0g/s 4332Kp/s 4332Kc/s 4332KC/s 207832jr..207115ag
4 0g 0:00:02:18  0g/s 4294Kp/s 4294Kc/s 4294KC/s nnhnccc..nnhnty9
3 0g 0:00:02:18  0g/s 4377Kp/s 4377Kc/s 4377KC/s 7wcmdo..7wc42c
4 0g 0:00:08:40  0g/s 4608Kp/s 4608Kc/s 4608KC/s apidl1331..apidl1737
1 0g 0:00:08:40  0g/s 4600Kp/s 4600Kc/s 4600KC/s SJ1xir..SJ1xdm
2 0g 0:00:08:40  0g/s 4597Kp/s 4597Kc/s 4597KC/s fiqolgo..fiqrmts
3 0g 0:00:08:40  0g/s 4589Kp/s 4589Kc/s 4589KC/s 2mc1n68..2mc2lw9
2 0g 0:00:10:40  0g/s 4610Kp/s 4610Kc/s 4610KC/s 8f9pp8..8fSMUF
3 0g 0:00:10:40  0g/s 4603Kp/s 4603Kc/s 4603KC/s salng87..salnj26
4 0g 0:00:10:40  0g/s 4634Kp/s 4634Kc/s 4634KC/s slpjrumu..slpjrp28
1 0g 0:00:10:40  0g/s 4617Kp/s 4617Kc/s 4617KC/s lussbedow..lussbiras
Waiting for 3 children to terminate
Session aborted

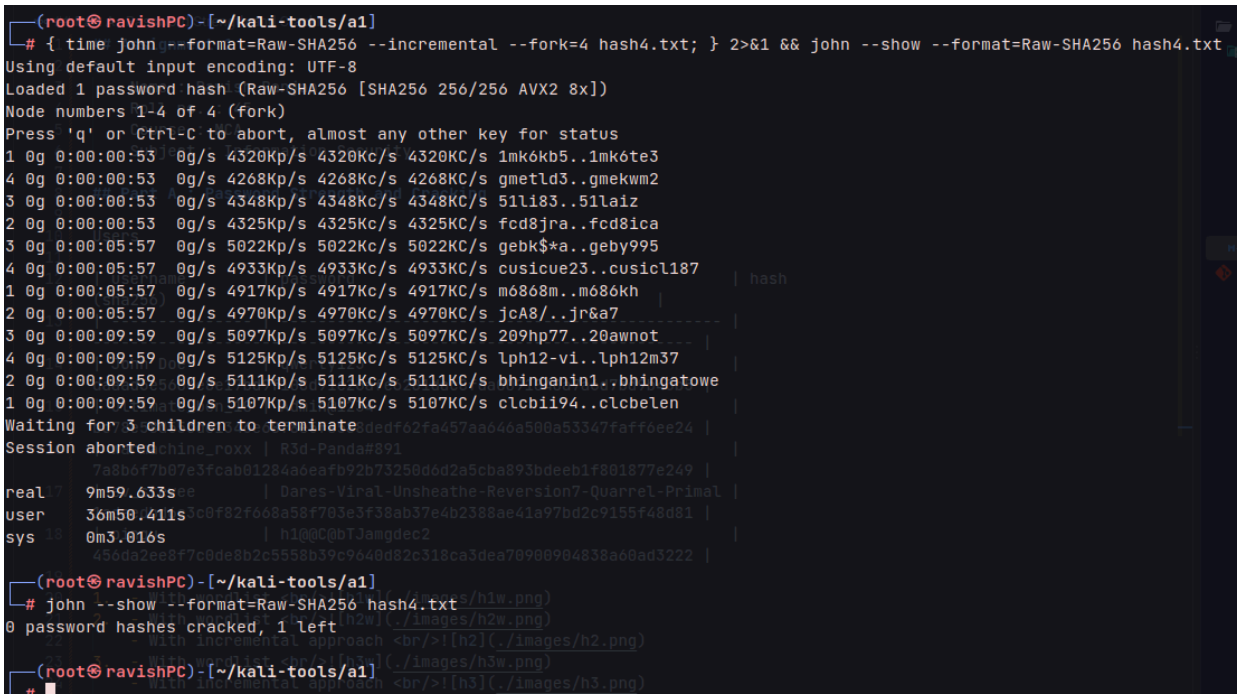
real    10m40.628s
user    35m21.837s
sys     0m4.448s

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# john --show --format=Raw-SHA256 hash3.txt
0 password hashes cracked, 1 left
```

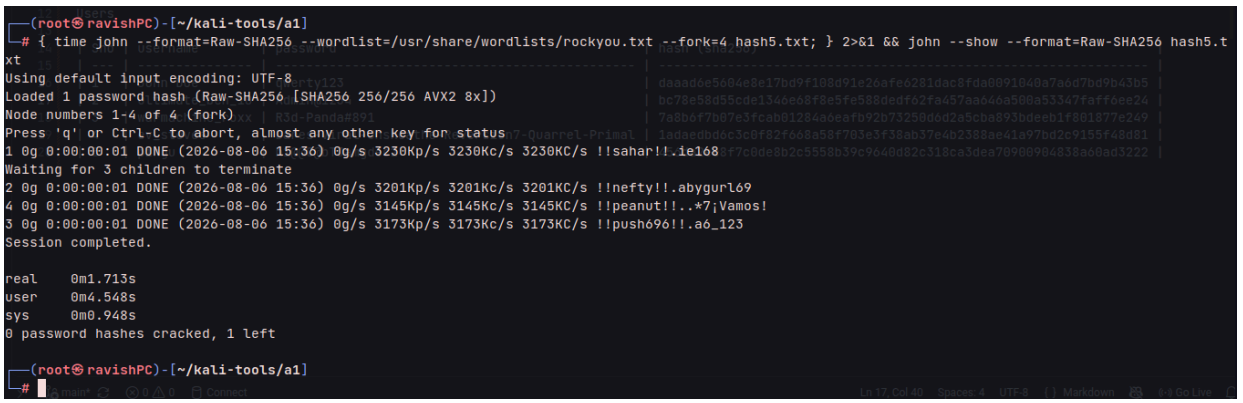
4.
   - With wordlist



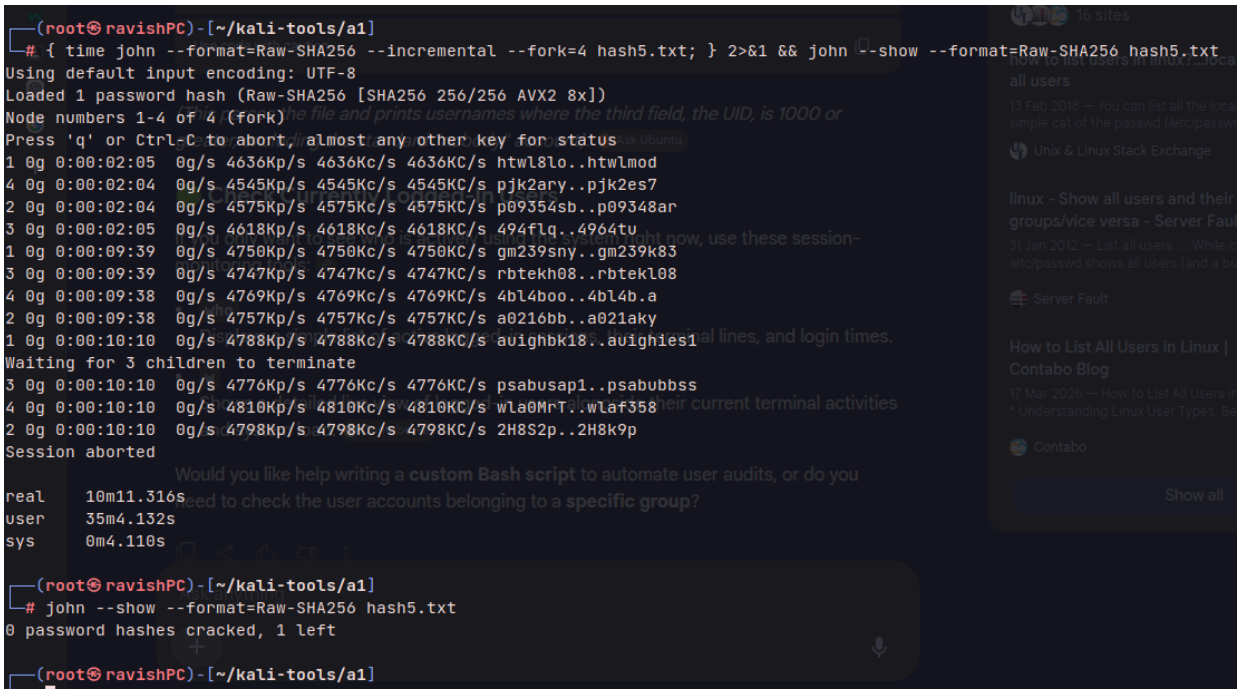
   - With incremental approach



5.
   - With wordlist



- With incremental approach



password	complexity	cracked	time	method
qwerty123	very low	yes	0.304s	wordlist
Admin@1234	low	no	checked till (10 mins)	none worked
R3d-Panda#891	mid	no	checked till (10 mins)	none worked
Dares-Viral-Unsheathe-Reversion7-Quarrel-Primal	high	no	checked till (10 mins)	none worked
h1@@C@bTJamgdec2	very high	no	checked till (10 mins)	none worked

The experiment demonstrates a clear relationship between password complexity and resistance to password cracking attacks

- The very low complexity password (qwerty123) was cracked almost instantly because it is a common password present in the rockyou.txt wordlist.
- The low complexity password (Admin@1234) was not cracked despite containing a common word and predictable number pattern. This indicates that although the password is relatively weak, it was not present in the tested wordlist, and the incremental attack did not discover it within the 10-minute time limit.
- The medium complexity password (R3d-Panda#891) combines uppercase and lowercase letters, numbers, symbols, and multiple words, increasing its search space and making it resistant to both attack methods during the test period.
- The high complexity password (Dares-Viral-Unsheathe-Reversion7-Quarrel-Primal) has a very long length and high entropy. These characteristics make exhaustive searching computationally expensive, preventing it from being cracked within the allotted time.
- The very high complexity password (h1@@C@bTJamgdec2) contains a random combination of uppercase letters, lowercase letters, numbers, and special characters. Such randomness significantly increases the number of possible combinations, making it highly resistant to dictionary and brute-force attacks.

Part B : Discretionary Access Control (DAC) on Linux

1. creating users

```
useradd userA
useradd userB
passwd userA
passwd userB
```

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:997:997:systemd Network Management:/:/usr/sbin/nologin
messagebus:x:996:996:System Message Bus:/nonexistent:/usr/sbin/nologin
tcpdump:x:995:995:tcpdump:/nonexistent:/usr/sbin/nologin
userA:x:1000:1000::/home/userA:/bin/sh
userB:x:1001:1001::/home/userB:/bin/sh

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# passwd userA
New password:
Retype new password:
passwd: password updated successfully

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# passwd userB
New password:
Retype new password:
passwd: password updated successfully
```

2. Demonstrate, with commands and output, how chmod, chown, and setfacl/getfacl can be used to: - allow userA read/write but userB read-only access.

- creating directory and giving userA access

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# mkdir -p /labshare

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# touch /labshare/test.txt

┌──(root㉿ravishPC)-[~/kali-tools/a1]
```

```
└─# ls /labshare/
test.txt

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# chown userA:userA /labshare/

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# chown userA:userA /labshare/test.txt

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# ls -ld /labshare
drwxr-xr-x 2 userA userA 4096 Aug 6 16:22 /labshare
```

- demonstrating chmod with different acccess levels

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# chmod 755 /labshare/

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# chmod 644 /labshare/test.txt

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# ls -ld /labshare/
drwxr-xr-x 2 userA userA 4096 Aug 6 16:22 /labshare/

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# ls -l /labshare/
total 0
-rw-r--r-- 1 userA userA 0 Aug 6 16:22 test.txt
```

3. Demonstrating chown

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# chown userA:userA /labshare/test.txt

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# ls -l /labshare/test.txt
-rw-r--r-- 1 userA userA 0 Aug  6 16:22 /labshare/test.txt
```

4. Part (a) userA Read/Write, userB Read Only

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# getfacl /labshare/test.txt
getfacl: Removing leading '/' from absolute path names
# file: labshare/test.txt
# owner: userA
# group: userA
user::rw-
group::r--
other::r--

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# getfacl /labshare/
getfacl: Removing leading '/' from absolute path names
```

```
# file: labshare/
# owner: userA
# group: userA
user::rwx
group::r-x
other::r-x
```

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# su - userA

$ echo "Written by userA" >> /labshare/test.txt
$ cat /labshare/test.txt
Written by userA
$ exit

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# su - userB

$ cat /labshare/test.txt
Written by userA
$ echo "Hello from userB" >> /labshare/test.txt
-sh: 2: cannot create /labshare/test.txt: Permission denied
$ ls /labshare
test.txt
$
```

5. Part (b): Completely Deny userB Access

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# setfacl -m u:userB:--- /labshare

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# setfacl -m u:userB:--- /labshare/test.txt

┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# getfacl /labshare/
getfacl: Removing leading '/' from absolute path names
# file: labshare/
# owner: userA
# group: userA
user::rwx
user:userB:---
group::r-x
mask::r-x
other::r-x
```

```
┌──(root㉿ravishPC)-[~/kali-tools/a1]
└─# su - userB

$ ls /labshare
ls: cannot open directory '/labshare': Permission denied
$ cat /labshare/test.txt
cat: /labshare/test.txt: Permission denied
$ echo "Hi" >> /labshare/test.txt
```

```
-sh: 3: cannot create /labshare/test.txt: Permission denied
$
```

Report

Authentication and access control are fundamental security mechanisms that help preserve the confidentiality of information within an organization. Authentication verifies the identity of a user before granting access to a system, ensuring that only legitimate users can log in. However authentication alone is not sufficient. Access control determines what authenticated users are allowed to view, modify, or execute based on their roles and permissions. This prevents unauthorized users from accessing sensitive files, modifying critical data, or performing actions beyond their responsibilities. In Linux systems, mechanisms such as file ownership like in chown, permission bits like in chmod, and Access Control Lists like with setfacl provide fine-grained control over who can access resources. Together, authentication and access control reduce the risk of data breaches, insider threats, and accidental disclosure of confidential information.

To strengthen password security, organizations should implement a strong password policy that requires passwords to be at least 14–16 characters long and encourages the use of passphrases or randomly generated passwords instead of predictable words or patterns. Additionally, organizations should enforce multi-factor authentication for all user accounts. Even if a password is compromised through phishing or brute-force attacks, MFA provides an additional verification step, significantly reducing the likelihood of unauthorized access. Regular security awareness training and periodic password audits can further improve the overall security posture of the organization.